

Module 04: Feature Engineering II - Automated Selection and Adversarial Validation

Cybernauts 2026 Datathon Campaign

May 2026

Abstract

As you engineer hundreds of features, your dataset becomes cluttered with noise and redundancy. This note covers the critical "Pruning Phase"—how to systematically drop features that don't help, and a secret Grandmaster technique called "Adversarial Validation" to ensure your model doesn't fail when it hits the hidden test set.

Contents

1	Systematic Pruning: Cleaning the Feature Forest	2
1.1	Feature Importance Filtering	2
1.2	Dropping Collinear Features	2
2	Adversarial Validation: Detecting Data Drift	2
2.1	The Concept	2
2.2	Interpreting the Results	2
3	Practice Problems	3

1 Systematic Pruning: Cleaning the Feature Forest

Engineering 200 features is easy; keeping the right 50 is the hard part.

1.1 Feature Importance Filtering

The fastest way to rank features is by running a baseline **LightGBM** model with default parameters. *Why?* Tree-based models calculate **Gain** and **Split** importance, telling you exactly which columns provided the most signal.

```
1 import lightgbm as lgb
2 model = lgb.LGBMRegressor().fit(X, y)
3 importance_df = pd.DataFrame({'feature': X.columns, 'importance': model.
4 feature_importances_})
5 importance_df = importance_df.sort_values(by='importance', ascending=False)
6 # Strategy: Drop the bottom 30% of features
```

1.2 Dropping Collinear Features

Redundant features add noise and slow down training. If two features have a correlation > 0.95 , they are essentially the same.

The Selection Rule

Identify pairs of highly correlated features. Keep the one that has the **strongest relationship to the target** and drop the other.

2 Adversarial Validation: Detecting Data Drift

This is a high-level competitive technique to see if your training data and test data come from different distributions.

2.1 The Concept

1. Label all your training rows as 0 and all test rows as 1.
2. Combine them into one big dataset (dropping the actual target).
3. Train a model to predict: **"Is this row from the Test Set?"**

2.2 Interpreting the Results

- **AUC ≈ 0.50 :** The model can't tell the difference. Your data is stable.
- **AUC > 0.70 :** The model can identify test rows. This means some features behave differently in the test set.

Action Plan

If the AUC is high, check the feature importance of your Adversarial Model. The features at the top are "leaking" information about the test set. **Drop them immediately** to avoid a massive score drop on the private leaderboard.

3 Practice Problems

P1: You have engineered 150 features. You run a baseline model and find that 40 features have an importance score of exactly 0. What should you do?

Answer: Drop them. They are pure noise and never used for a single split in the tree model.

P2: Your Adversarial Validation model has an AUC of 0.85. The top feature is "Transaction_Date". What does this tell you?

Answer: The test data likely comes from a different time period than the training data. This feature allows the model to "memorize" the time gap. You should probably drop it or transform it into a relative feature (e.g., "Days Since Start").